Lesson 12 -> Bloom filters

1️⃣ What Is a Bloom Filter?

A **Bloom filter** is a **probabilistic data structure used to test whether an element is in a set**.

Key properties:

- It can say:
  - ✅ "Definitely not present"
  - ⚠️ "Probably present"
- It **never gives false negatives**
- It **can give false positives**

That means:

- If Bloom filter says "not present" → guaranteed true.
- If Bloom filter says "present" → might be wrong.

Core Idea (Intuition)

Instead of storing actual values, Bloom filters:

- Use a **bit array**
- Use multiple **hash functions**
- Mark bits corresponding to hash positions

It trades:

- **Perfect accuracy**
  for
- **Huge memory savings**

2️⃣ Why Do We Need Bloom Filters?

In large systems:

- You may need to check membership millions/billions of times.
- Storing full sets may be too expensive.

- Disk lookups are slow.

Bloom filters help answer:

> "Is this value worth looking up in expensive storage?"

Classic Problem

Suppose:

- You have 1 billion user IDs stored in a database.
- Every request checks if user exists.
- Disk lookup is expensive.

Instead:

1. Check Bloom filter (in memory)
2. If it says "not present" → skip DB
3. If it says "maybe present" → query DB

Result:

- Massive performance improvement.

3️⃣ Advantages & Disadvantages

Advantages

✅ 1. Extremely Memory Efficient

Stores huge sets in tiny memory.

✅ 2. Very Fast

Only hash computations + bit checks.

✅ 3. No False Negatives

If it says "not present", it's guaranteed.

✅ 4. Great for Read-Heavy Systems

Perfect for pre-checks before DB calls.

Disadvantages

❌ 1. False Positives

May say “present” even when not.

❌ 2. Cannot Remove Elements (Standard Version)

No deletions unless using counting Bloom filters.

❌ 3. Cannot Retrieve Items

Only membership check.

❌ 4. Error Rate Increases with Load

As more elements inserted → more false positives.

4️⃣ Use Cases of Bloom Filters

Bloom filters are widely used in:

1️⃣ Databases (Very Common)

Cassandra

Uses Bloom filters to:

- Avoid unnecessary disk reads.

Before reading SSTable:

- Check Bloom filter.
- If negative → skip file.

2️⃣ Web Caching

Example:

- CDN checks if object exists in cache.
- Bloom filter avoids disk/network calls.

3️⃣ Distributed Systems

Example:

- Avoid sending requests to nodes that don't have data.

4️⃣ Password Blacklists

- Check if password is compromised.
- Efficient storage of large leaked password sets.

5️⃣ Blockchain

- Quickly check if transaction might be in a block.

5️⃣ Example of Bloom Filter (Conceptual Walkthrough)

Let's build intuition with a simple example.

Step 1: Create Bit Array

Size = 10 bits:
[0 0 0 0 0 0 0 0 0 0]

Step 2: Add "apple"

Apply 3 hash functions:
hash1("apple") → 2
hash2("apple") → 5
hash3("apple") → 7

Set those bits:
[0 0 1 0 0 1 0 1 0 0]

Step 3: Add "banana"

Hashes:
1, 5, 9
Set bits:
[0 1 1 0 0 1 0 1 0 1]

Step 4: Check "orange"

Hashes:
2, 5, 8
Check bits:

- bit 2 = 1
- bit 5 = 1
- bit 8 = 0

Since one bit is 0 →
👉 Definitely not present.

Step 5: Check “grape”

Hashes:
1, 5, 7

All bits are 1 →
👉 Probably present (might be false positive).

6️⃣ How Bloom Filters Are Useful in Data Science

Now the interesting part.

1️⃣ Feature Store Optimization

Before querying large feature store:

- Check Bloom filter
- Avoid unnecessary lookups

2️⃣ Deduplication in Data Pipelines

Example:

- Streaming logs
- Avoid processing duplicate records

Bloom filter:

- Fast membership test
- Memory efficient

3️⃣ Training Data Cleaning

Checking if:

- ID already processed
- Record already seen

Very useful in large-scale pipelines.

4️⃣ Large-Scale User Systems

Recommendation systems:

- Quickly check if user interacted before
- Reduce DB load

5️⃣ Web-Scale ML Systems

Search engines:

- Quickly filter non-existing terms
- Pre-filter candidate sets

7️⃣ Architect-Level Perspective

Bloom filters are used when:

- Data is huge
- Memory is limited
- Some error tolerance is acceptable
- You need fast negative filtering

They are **performance accelerators**, not truth sources.

8️⃣Key Trade-Off Formula (Advanced Insight)

False positive rate depends on:

- Number of elements (n)

- Bit array size (m)
- Number of hash functions (k)

There is an optimal:

$k = (m / n) * \ln(2)$

This minimizes false positives.

9️⃣ Interview-Ready Summary

- Bloom filter is a probabilistic data structure for membership testing.
- It guarantees no false negatives but allows false positives.
- Used to reduce expensive lookups in large systems.
- Extremely memory efficient.
- Widely used in distributed databases like Cassandra.